CS 3510 Section B: Design & Analysis of Algorithms September 19th 2024

Problem Set 4: Topological Sort, MSTs, Shortest Paths

Devin Fromond Due: September 27th 2024

- Please type your solutions using LaTeX or any other software. Handwritten solutions will not be accepted.
- Your algorithms must be in plain English & mathematical expressions, and the pseudo-code is optional. Pseudo-code, without sufficient explanation, will receive no credit.
- Unless otherwise stated, all logarithms are to base two.
- If we ask for a specific running time, a correct solution achieving it will receive full credit even if a faster solution exists.

Problem 1: House Placement (25 points)

As a suburban planner, you need to place k houses in Ladhaland numbered 1 to k in a $k \times k$ grid. However, you are given two lists of house pairs labeled `rowConstraints` and `colConstraints` of length m and n respectively. For a pair (a, b) in `rowConstraints`, there is a requirement that house a must be in a row strictly above the row that house b is in. For a pair (a, b) in `colConstraints`, there is a requirement that house a must be in a column strictly to the left of the column that house b is.

Design an algorithm that returns the row and column coordinates for each of the k houses given the number of houses k, `rowConstraints`, and `colConstraints`. If it is impossible to fulfill all the requirements in `rowConstraints` and `colConstraints`, return `"Impossible"`. Provide a correctness and runtime analysis.

Solution:

- *Algorithm:* The algorithm we wish to use consists of three primary steps: (1) creating directed graphs for *rowConstraints* and *colConstraints*, (2) performing Topological sort to model this problem on each directed graph, where we return "*Impossible*" if a cycle is found, and (3) return the positions of the houses using the *rowConstraints* graph as row-input and *colConstraints* as column-input.

  First, we wish to create two directed graphs. We wish to add a directed edge from a to b in the form (a, b). Next, we will perform topological sort on the both graphs, returning "*Impossible*" if a cycle is found. The output of this sort will return the house number in the form of an array-like structure, such that it can easily be iterated over. After topological sort is called, we do not require the complexity of directed edges (or edges at all) if no cycle has been detected. As such, an array will be the quickest method to place all houses in the grid. For example, if the row array returned by topological sort is $[4, 5, 1]$, then house 4 will be on row 0, house 5 on row 1, and house 1 will be on row 2. If a house is not present in the row array, then it does not have a constraint. As such, we can add all houses that do not have a constraint to the end of the array in no particular order. We will do the same with the column array. Finally, we return the coordinates for each house based off of the array.
- *Correctness:* The algorithm maintains all constraints are considered via topological sort. Furthermore, if the requirements are unable to be fulfilled, we know there exists a cycle since there will be two directed edges to a particular house. Since we check for a cycle and return Impossible if one is detected, we know we correctly consider all constraints. We wish to consider the constraints first and the houses without constraints second. As such, we place the houses without constraints at the end of their respective array. This ensures all houses are placed in the array and therefore all house coordinates are returned.
- *Time Complexity Analysis:* We observe the graph creations are directly dependent on the number of constraints. Therefore, we have $O(n + m)$, where m is the number of row constraints and n is the number of column constraints. Furthermore, we have

$O(n+k)$ and $O(m+k)$, respectively, for performing topological sort on the row graph and column graph since there are k vertices and n/m edges. This time complexity considers the time it takes to add all non-constrained houses to the end of the array through k vertices. Finally, we must traverse over the arrays to return the the pairs which takes $O(2k)$. Ultimately, we arrive at our final time complexity of $O(m+n+k)$.

Problem 2: Populate Banana Centers (25 points)

There are n banana centers in Decatur numbered 1 to n. Your goal is to provide a source of bananas for all the banana centers by building banana farms and conveyor belts to transport bananas.

You have two options for each banana center i: we can either build a farm inside the center with cost f_i, or we can build a conveyor belt from another center. You are given an array `conveyors` where `conveyors`$[i] = [u, v, c]$ which denotes that the cost of building a conveyor belt from centers u to v is c. Conveyor belts are bidirectional and there may be multiple valid connections between the same two centers with different costs.

Design an algorithm to return the minimum cost of populating all banana centers with farms. Provide a runtime and correctness analysis.

Solution:

- *Algorithm:* We find this problem to be similar to the applications of Kruskal's algorithm for a Minimum Spanning Tree. We begin by treating each banana center as a node then build edges using the *conveyors* array. Since banana centers can have a farm inside the center, we initialize a node with a self-loop with a weight of f_i.

  We then perform Kruskal's algorithm - as described in lecture - over the graph, where we initialize a $total_cost$ variable before the second for loop, updating the total cost after the *union* call, then finally returning $total_cost$ after the for loop has completed (which is the end of the algorithm).

  It is vital to note that Kruskal's algorithm needs one major modification. We do not require a banana center to be connected to other banana centers - it is possible for a banana center to be connected to itself and to no other nodes. Therefore, we wish to adjust the second for loop such that it checks first if $u == v$. If this it $True$, then it is a self-farm and f_i will be added to $total_cost$. Else, we continue the normal logic of checking if $find(u) \neq find(v)$ and performing the next steps as described in lecture (and as mentioned above).
- *Correctness:* Kruskal's Algorithm guarantees an MST is found, which ensures the minimum cost to either build farms or connect centers with conveyor belts. However, as previously noted, we do not wish to necessarily connect a banana center. Since we modified Kruskal's algorithm to properly ignore unions for banana centers that have a farm inside of them if not necessary (i.e. if there is no conveyor belt), we guarantee correctness by finding the path with minimum edges to all banana centers - where applicable. This returns the minimum cost of populating all banana centers with farms, as desired, since we consider the fact banana centers can have a farm in them.
- *Time Complexity Analysis:* We observe the time complexity for creating the graph of n banana centers to be composed of two things: connecting banana center to themselves (self-loop) and having a conveyor between centers. Therefore, we view the time com-

plexity to be $O(n)$ and $O(m)$, respectively where m denotes the number of conveyors since we must create an edge for each conveyor in the *conveyors* array. Therefore, creating the graph yields a time complexity of $O(n+m)$. We let E represent the edges of the graph. Therefore, we have $O(E)$.

To sort the edges, we arrive at a time complexity of $O(Elog(E))$. Finally, we observe union-find operations take $O(Elog(n))$ in total, where n is the number of banana centers. Therefore, since $E > n$, we arrive at our final time complexity of $O(Elog(E))$.

Problem 3: Currency Exchange (25 points)

You are given a list of currencies in Middle Earth: $C = [c_1, c_2, c_3, \ldots, c_n]$. The Middle Earth Bank releases a list of valid exchanges between currencies as a list R of size m where entry $R[i] = [c_u, c_v, e]$ denotes that 1 unit of currency c_u yields e units of currency c_v. For example, at the time of writing this exam, 1 U.S. dollar is equivalent to 0.93 Euros, so you might see an entry [USD, Euro, 0.93], if Middle Earth had USDs and Euros. Given 1 unit of a start currency c_s and a target currency c_t, design an algorithm to compute the most amount of units of c_t you could obtain by converting through valid exchanges. Provide a runtime and correctness analysis.

Note: it is not necessarily true that if you can convert from c_u to c_v, you can convert from c_v to c_u. Additionally, you can assume it's not possible to generate infinite money using valid exchanges.

Solution:

- *Algorithm:* We recognize given 1 unit of c_s, we wish to maximize c_t. We let each currency in C be a node and the directed edge between c_u and c_v be determined by R. We find using e as provided by the list R requires the algorithm to utilize multiplication, which is not very intuitive for finding the maximum distance. Instead, we wish to take the *log* of the edge weight as determined by R. This will allow us to add distances instead of multiplying them. As a result of taking the *log*, we recognize some distances may now be negative. As a result, we are unable to use Dijkstra's algorithm and instead, we wish to use Bellman-Ford's algorithm as it works with edge weights.

  We wish to utilize the Bellman-Ford algorithm as described in class, with a couple modifications. We begin by initializing an array named *dist* of size $len(C)$, each with an initial size of $-inf$. We then initialize the starting currency to 0, since $log(1) = 0$. We then utilize the embedded for-loop structure for Bellman-Ford, as described in class. Inside of the second for loop, we set the current index of the *dist* array to $max(dist(v), dist(u) + log(e)$, where e is the exchange rate (or edge weight) between u and v. Finally, outside of the for loop, we return "no path" if the value of $c_t = -inf$; else, we return $exp(dist[c_t])$ to undo the logarithm (meaning there is a path between c_s and c_t.
- *Correctness:* Correctness is guaranteed through the altered Bellman-Ford algorithm, which successfully finds the maximum between two nodes, given the potential for negative edge weights. Through the logarithm property, we know the maximum exchange weight is calculated, which is guaranteed to not exist within a loop to provide infinite money. We know the correct value is returned since the distances are updated in the *dist* array.
- *Time Complexity Analysis:* We observe the algorithm takes $O(nm)$ time where n is the number of currencies and m is the number of valid exchanges. This is because we utilize two for loops: one that iterates through $n - 1$ and the other that iterates over

every edge. Since these for loops are embedded, we multiply the time for each and arrive at our final answer of $O(nm)$.

Problem 4: Research Stations (25 points)

There are several research stations marked with numbers from 1 to n. These stations are linked by m cables, each having a specific time t it takes for a message to travel through. Each research station is configured to simply split and relay a received message from one cable to all the other cables.

Due to the poor quality of the wires, a message that takes more than t_{limit} time to transmit will simply be lost and corrupted. For example, if station 1 is linked by a 5 second cable to station 2 and station 2 is linked by another 5 second cable to station 3, station 3 will not be able to receive a message from station 1 if $t_{limit} < 10$. Therefore, we need to find the research station that can transmit a message that will be received by the most stations. This station will be designated as the central station.

Design an algorithm to find this central station given the number of stations n, the m cables each having specific transmission time t, and the message transmission time limit t_{limit}. Provide a runtime and correctness analysis.

Solution:

- *Algorithm:* To solve the problem, we wish to utilize a modified BFS algorithm. We wish to find the central research station that can transmit messages to the maximum number of other stations within a specified time limit.

  We begin by initializing two global variables: *maxCount* to track the maximum number of stations reached and *centralStation* to keep track of the current central station. Next, we construct the graph, representing each research station as a node and linking two stations with an edge if they are connected by a cable.

  For each of the n research stations, we will perform the following sequence using BFS to determine reachability:

  For each starting station, we initialize a variable named *count* to denote the number of reachable stations. We create a list called *visited* to keep track of which stations have already been processed. Additionally, we initialize a queue and place the current station it it with a initial cumulative transmission time of 0.

  While the queue is not empty, we dequeue the front, which consists of the current station and the cumulative time taken to reach it. If the current time exceeds the specified t_{limit}, we skip any further processing for that station.

  If the current time is within the t_{limit}, we increment *count*, indicating that this station is reachable. We then explore all neighboring stations connected by edges, updating the cumulative transmission time for each neighbor. If a neighboring station has not been visited and can be reached within the time limit, we mark it as visited and enqueue it for further exploration.

  After processing all reachable nodes for the current starting station, if *count* exceeds *maxCount*, we update *maxCount* and set *centralStation* to the current station being processed.

After completing this process for all stations, we return the *centralStation*, which represents the station capable of transmitting messages to the maximum number of other stations within the specified time limit.

- *Correctness:* By performing BST from each station, we explore all stations and determine the maximum number that station can explore. This ensures no station nor connection is overlooked. Furthermore, we ensure the station is reachable by comparing the current time to t_{limit}. Thereby only allowing valid stations to update the count. Furthermore, the algorithm updates the central station if it finds a station with a higher count, thereby ensuring the most optimal central station is returned. The overall correctness is guaranteed by BFS and the aforementioned properties.
- *Time Complexity Analysis:* We view the construction of the adjacency list takes $O(m)$ time, where m is the number of cables. Additionally, we know BFS traversal takes $O(n+m)$ time since every station and cable is visited once per BFS iteration. Since BFS is run for n research stations, we observe $O(n(n+m)+m)$. Therefore, we have a final time complexity of $O(n(n+m))$.